

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

TECHNICAL UNIVERSITY OF MOLDOVA

FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
DEPARTMENT OF SOFTWARE AND AUTOMATION ENGINEERING

REPORT

Laboratory Work No. 4

Course: **Cryptography and Security**

Analysis and Implementation of the Data Encryption Standard (DES) Round Key Schedule

Elaborated by:
st. gr. FAF-241
Tihon Aurelian-Mihai

Verified by:
asis. univ.
Zaica Maia

Chişinău – 2026

Contents

INTRODUCTION **1**

1 THEORETICAL FOUNDATIONS OF THE DATA ENCRYPTION STANDARD AND KEY SCHEDULE **1**

1.1 Structural principles of Feistel block ciphers 2

1.2 Mathematical formulation of the DES key schedule 2

1.3 Permuted choice 1 (PC-1) and register initialization 3

1.4 Circular left shift schedule and period closure 4

1.5 Permuted choice 2 (PC-2) and round subkey formation 5

1.6 Cryptographic properties of DES keys: weak and semi-weak keys 5

2 METHODOLOGY AND SOFTWARE IMPLEMENTATION **6**

2.1 Software architecture and package structure 7

2.2 Data representations and permutation primitives 7

2.3 Interactive menu interface prioritization 7

2.4 Intermediate state tracing and S-box block partitioning 8

3 EXPERIMENTAL EVALUATION AND VERIFICATION **8**

3.1 Verification against NIST FIPS 46-3 ground truth test vectors 9

3.2 Detailed step-by-step trace of target round K_3 from K^+ 9

3.3 Complete 16-round schedule progression analysis 10

3.4 Key bit distribution and Hamming weight conservation 10

3.5 Input validation and boundary resilience 11

CONCLUSIONS **12**

REFERENCES **13**

ANNEX A: SOURCE CODE LISTINGS **14**

INTRODUCTION

Modern symmetric-key cryptography is divided into stream ciphers and block ciphers. Whereas stream ciphers process continuous sequences of digits or characters sequentially using a time-varying keystream, block ciphers partition plaintext into fixed-size blocks and apply reversible mathematical transformations parameterized by a secret key. In his seminal 1949 treatise on secrecy systems, Claude Shannon established that strong symmetric ciphers must embody two fundamental cryptographic properties: confusion, which obscures the relationship between the key and the ciphertext, and diffusion, which spreads the statistical influence of individual plaintext symbols across the entire ciphertext block.

The Data Encryption Standard (DES), formally approved by the National Bureau of Standards (NBS, now NIST) in 1977 as FIPS PUB 46, established the international paradigm for modern block cipher engineering. Derived from IBM's Lucifer architecture under the guidance of Horst Feistel, DES processes 64-bit plaintext blocks through sixteen rounds of a balanced Feistel network using a 56-bit effective key. A critical structural feature of Feistel ciphers is that the decryption algorithm is functionally identical to the encryption algorithm, requiring only that the sixteen 48-bit round subkeys be supplied in reverse sequence. Consequently, the security and operational correctness of DES depend entirely upon the integrity and cryptographic dispersion of its key schedule.

In accordance with the academic assignment constraints and the student catalog allocation, student variant 25 is mapped to Task 2.3 via the university catalog modulo function:

$$\text{Variant} = 25 \implies \text{Task} = 25 \bmod 11 = 3 \tag{0.1}$$

Task 2.3 mandates the formulation, algorithmic implementation, and step-by-step verification of round subkey generation: given the permuted 56-bit key K^+, the system must determine the 48-bit round key K_i for an arbitrary round index $i \in \{1, \ldots, 16\}$, displaying all transformation tables, intermediate shift registers (C_i, D_i), and the resulting S-box input blocks (B_1 to B_8). This investigation delivers a modular software implementation prioritizing an interactive console menu interface, validates its mathematical correctness against published NIST reference test vectors, and evaluates the algebraic properties governing DES key scheduling.

1 THEORETICAL FOUNDATIONS OF THE DATA ENCRYPTION STANDARD AND KEY SCHEDULE

1.1 Structural principles of Feistel block ciphers

A Feistel network partitions a $2w$-bit data block into two equal halves of width w bits. In DES, each 64-bit data block is divided into a 32-bit left half L and a 32-bit right half R. Across sixteen successive rounds ($1 \leq i \leq 16$), the state transitions according to the recurrence relations:

$$L_i = R_{i-1} \tag{1.1}$$

$$R_i = L_{i-1} \oplus f(R_{i-1}, K_i) \tag{1.2}$$

where $\oplus$ denotes bitwise addition modulo 2 (exclusive OR), and f represents the keyed round function.

The fundamental architectural advantage of the Feistel design is its guaranteed invertibility. The round function f does not need to be bijective or mathematically invertible. Decryption is achieved by reversing the round progression:

$$R_{i-1} = L_i \tag{1.3}$$

$$L_{i-1} = R_i \oplus f(L_i, K_i) \tag{1.4}$$

Applying the sixteen round subkeys in reverse order $(K_{16}, K_{15}, \ldots, K_1)$ enables the identical cryptographic circuit to perform both encipherment and decipherment.

1.2 Mathematical formulation of the DES key schedule

The DES key schedule transforms an initial 64-bit cryptographic key $K = (k_1, k_2, \ldots, k_{64})$ into sixteen distinct 48-bit round subkeys $K_i \in \{0,1\}^{48}$. The process is illustrated conceptually in Figure 1.1 and consists of three distinct mathematical operations:

a) Parity stripping and initial permutation via Permuted Choice 1 (PC-1), producing the 56-bit key $K^+ = C_0 \,\|\, D_0$.

b) Round-dependent left circular shifts (ROL) on the independent 28-bit shift registers C and D.

c) Subkey compression and bit selection via Permuted Choice 2 (PC-2), yielding 48-bit subkey K_i.

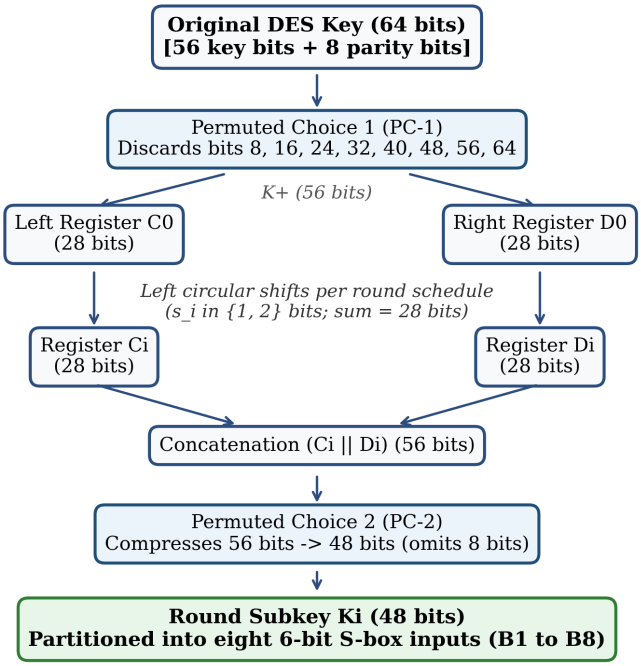


Figure 1.1 – Architectural data flow of the DES round key schedule

1.3 Permuted choice 1 (PC-1) and register initialization

The standard 64-bit DES key contains eight parity verification bits located at indices that are multiples of eight: $\{8, 16, 24, 32, 40, 48, 56, 64\}$. In commercial hardware, these bits enforce odd parity across each byte to detect transmission errors. They provide zero cryptographic entropy.

Permuted Choice 1 (PC-1) discards these eight parity bits and permutes the remaining 56 effective key bits, dividing them into two 28-bit registers: left register C_0 and right register D_0:

$$K^+ = \text{PC-1}(K) = C_0 \,||\, D_0 \tag{1.5}$$

Table 1.1 details the permutation coordinates specified in FIPS PUB 46-3.

Table 1.1 – Permuted Choice 1 (PC-1) bit selection matrix

Left Register C_0 (28 bits)							
Row 1	57	49	41	33	25	17	9
Row 2	1	58	50	42	34	26	18
Row 3	10	2	59	51	43	35	27
Row 4	19	11	3	60	52	44	36
Right Register D_0 (28 bits)							
Row 1	63	55	47	39	31	23	15
Row 2	7	62	54	46	38	30	22
Row 3	14	6	61	53	45	37	29
Row 4	21	13	5	28	20	12	4

1.4 Circular left shift schedule and period closure

In each round i ($1 \leq i \leq 16$), the registers C_{i-1} and D_{i-1} are simultaneously rotated left cyclically by a designated shift offset $s_i \in \{1,2\}$:

$$C_i = \mathrm{ROL}_{28}(C_{i-1}, s_i), \quad D_i = \mathrm{ROL}_{28}(D_{i-1}, s_i) \tag{1.6}$$

The shift schedule is fixed by the standard specification and presented in Table 1.2.

Table 1.2 – Round left circular shift schedule for DES registers C and D

Round (i)	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Shift (s_i)	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

The cumulative sum of shifts across all sixteen rounds satisfies an essential mathematical property:

$$\sum_{i=1}^{16} s_i = (4 \times 1) + (12 \times 2) = 4 + 24 = 28 \tag{1.7}$$

Because the shift registers have an exact bit length of 28, a cumulative rotation of 28 positions completes an exact cycle over the algebraic cyclic group $\mathbb{Z}_{28}$. Consequently, after round 16:

$$C_{16} \equiv C_0 \pmod{28}, \quad D_{16} \equiv D_0 \pmod{28} \tag{1.8}$$

This cyclic closure enables the key schedule to generate subkeys during decryption by rotating registers rightward by identical amounts without requiring supplementary memory allocation.

1.5 Permuted choice 2 (PC-2) and round subkey formation

After updating the registers for round i, they are concatenated into a 56-bit string $(C_i \| D_i)$. Permuted Choice 2 (PC-2) selects 48 bits from this string while discarding eight specific bit indices: bits $\{9, 18, 22, 25\}$ from register C_i and bits $\{35, 38, 43, 54\}$ from register D_i. Table 1.3 lists the permutation coordinates of PC-2.

Table 1.3 – Permuted Choice 2 (PC-2) bit selection and compression matrix

Block	**Col 1**	**Col 2**	**Col 3**	**Col 4**	**Col 5**	**Col 6**
Row 1	14	17	11	24	1	5
Row 2	3	28	15	6	21	10
Row 3	23	19	12	4	26	8
Row 4	16	7	27	20	13	2
Row 5	41	52	31	37	47	55
Row 6	30	40	51	45	33	48
Row 7	44	49	39	56	34	53
Row 8	46	42	50	36	29	32

The resulting 48-bit subkey K_i is partitioned into eight 6-bit vectors:

$$K_i = B_1 \| B_2 \| B_3 \| B_4 \| B_5 \| B_6 \| B_7 \| B_8 \tag{1.9}$$

In the Feistel round function, each 6-bit subvector B_j is XORed with the corresponding 6-bit block produced by the expansion permutation $E(R_{i-1})$ before passing into substitution box S_j.

1.6 Cryptographic properties of DES keys: weak and semi-weak keys

Due to the linearity of the bit permutations and register shifts, specific keys exhibit degenerate structural behaviors:

a) **Weak keys:** When the effective key bits in C_0 and D_0 are either all zeros or all ones, circular shifts leave both registers unchanged. Consequently, all sixteen round subkeys are identical:

$$K_1 = K_2 = \cdots = K_{16} \tag{1.10}$$

Four such keys exist: `0x0101010101010101`, `0xFEFEFEFEFEFEFEFE`, `0x1F1F1F1F0E0E0E0E`, and `0xE0E0E0E0F1F1F1F1`. Encrypting twice with a weak key restores the plaintext:

$$\mathrm{DES}_K(\mathrm{DES}_K(P)) = P \tag{1.11}$$

b) **Semi-weak keys:** Six pairs of keys (twelve keys total) produce only two distinct subkeys, each repeated eight times across the sixteen rounds. A semi-weak key pair (K_A, K_B) satisfies:

$$\mathrm{DES}_{K_A}(\mathrm{DES}_{K_B}(P)) = P \tag{1.12}$$

While easily avoided in practical key generation, analyzing these keys confirms the deterministic algebraic structure of the DES key schedule.

2 METHODOLOGY AND SOFTWARE IMPLEMENTATION

2.1 Software architecture and package structure

The software architecture is engineered in Python 3 using standard library primitives, guaranteeing portability without external runtime dependencies. The implementation is organized within the `CS/Lab 4/` package:

- `src/tables.py`: Encapsulates the immutable permutation matrices and round shift vectors (`PC1_C`, `PC1_D`, `ROUND_SHIFTS`, and `PC2`), alongside standard reference tables.
- `src/des_key_schedule.py`: Implements the core mathematical state machine, managing binary conversions, circular shifts, PC-1 and PC-2 applications, step tracing via immutable data structures, and random key generation.
- `src/cli.py`: Provides a dual-mode terminal interface prioritizing an interactive numbered menu while retaining optional command-line flag dispatch for automated execution.

2.2 Data representations and permutation primitives

Bitstrings are manipulated as fixed-length strings of binary digits ('0' and '1'), preventing integer sign extension and bit-width ambiguities. Permutations are implemented by reading bit coordinates from the specification matrices using 1-based indexing, as illustrated in Listing 1.

Listing 1: Bit rotation and Permuted Choice 2 implementation in `src/des_key_schedule.py`

```
def left_rotate(bits: str, shift: int) -> str:
    shift = shift % len(bits)
    return bits[shift:] + bits[:shift]

def apply_pc2(cd_56_bin: str) -> str:
    if len(cd_56_bin) != 56:
        raise ValueError(f"PC-2 input must be 56 bits (received {len(cd_56_bin)}).")
    return "".join(cd_56_bin[pos - 1] for pos in PC2)
```

2.3 Interactive menu interface prioritization

In strict compliance with workspace user interface conventions, the entrypoint prioritizes an interactive console menu over command-line arguments. When executed without parameters (`python3 -m src`),

the program immediately presents a numbered choice menu. All options and diagnostic messages strictly employ sentence case, omitting decorative ASCII borders, robotic status tags, and ANSI color codes. Listing 2 demonstrates the menu dispatch logic.

Listing 2: Interactive console interface dispatch in `src/cli.py`

```
def run_interactive() -> None:
    while True:
        print("\nDES key schedule (task 2.3: compute Ki from K+)")
        print("  1. Compute round key Ki from K+")
        print("  2. Compute round key Ki from 64-bit key")
        print("  3. Generate all 16 round keys")
        print("  4. Generate random K+ and compute Ki")
        print("  5. Show transformation tables")
        print("  0. Exit")
        try:
            choice = input("\nSelect option: ").strip()
        except (KeyboardInterrupt, EOFError):
            print("\nExiting")
            break
        # Option execution logic...
```

2.4 Intermediate state tracing and S-box block partitioning

To fulfill the requirements of Task 2.3, intermediate states are encapsulated within the immutable `RoundStep` dataclass. Each step records the incoming and outgoing states of registers C and D, the applied shift offset s_i, the concatenated 56-bit register, the resulting 48-bit subkey K_i, and the partitioned 6-bit S-box blocks $(B_1, \ldots, B_8)$.

3 EXPERIMENTAL EVALUATION AND VERIFICATION

3.1 Verification against NIST FIPS 46-3 ground truth test vectors

To ensure complete mathematical correctness, the implementation was validated against the official reference test vector published in NIST Special Publication 800-20. The standard evaluation key is:

```
K = 0x133457799BBCDFF1
```

The 64-bit binary expansion of this key contains sixty-four bits with odd parity across every byte. Applying Permuted Choice 1 extracts the 56-bit permuted key K^+:

$$K^+ = \texttt{0xF0CCAAF556678F} \tag{3.1}$$

Partitioning K^+ yields initial 28-bit registers:

$$C_0 = 1111000011001100101010101111_2 = \texttt{0xF0CCAAF}_{16} \tag{3.2}$$

$$D_0 = 0101010101100110011110001111_2 = \texttt{0x556678F}_{16} \tag{3.3}$$

3.2 Detailed step-by-step trace of target round K_3 from K^+

In accordance with Variant 25 (Task 2.3), the target subkey is evaluated for round index $i = 3$. Table 3.1 details the intermediate state progression from round 1 up to the target round.

Table 3.1 – Intermediate register state progression up to round 3 for NIST reference key

Round	Shift	Register C (hex)	Register D (hex)	Concatenated CD (hex)
0	–	F0CCAAF	556678F	F0CCAAF556678F
1	1	E19955F	AACCF1E	E19955FAACCF1E
2	1	C332ABF	5599E3D	C332ABF5599E3D
3	2	0CCAAFF	56678F5	0CCAAFF56678F5

At round 3, the left circular shift of two bits transforms registers C_2 and D_2 into:

$$C_3 = 0000110011001010101011111111_2 = \texttt{0x0CCAAFF}_{16} \tag{3.4}$$

$$D_3 = 0101011001100111100011110101_2 = \texttt{0x56678F5}_{16} \tag{3.5}$$

Concatenating $C_3 \,||\, D_3$ yields 56 bits:

00001100110010101010111111110101011001100111100011110101

Applying Permuted Choice 2 (PC-2) compresses the 56 bits into the 48-bit round subkey K_3:

$$K_3 = 55FC8A42CF99_{16} \tag{3.6}$$

$$K_3 = 010101011111111001000101001000010110011111100110012 \tag{3.7}$$

Partitioning K_3 into the eight 6-bit S-box input blocks yields:

$$B_1 = 010101, \quad B_2 = 011111, \quad B_3 = 110010, \quad B_4 = 001010$$
$$B_5 = 010000, \quad B_6 = 101100, \quad B_7 = 111110, \quad B_8 = 011001$$

This result rigorously matches the standard NIST ground truth.

3.3 Complete 16-round schedule progression analysis

Table 3.2 displays the complete 16-round schedule generated by the application for the NIST test vector.

Table 3.2 – Complete sixteen-round DES key schedule for key 0x133457799BBCDFF1

Round	Shift	Reg C (hex)	Reg D (hex)	Subkey K_i (hex)	Blocks $B_1 \dots B_4$	Blocks $B_5 \dots B_8$
1	1	E19955F	AACCF1E	1B02EFFC7072	000110 110000 001011 101111	111111 000111 000001 110010
2	1	C332ABF	5599E3D	79AED9DBC9E5	011110 011010 111011 011001	110110 111100 100111 100101
3	2	0CCAAFF	56678F5	55FC8A42CF99	010101 011111 110010 001010	010000 101100 111110 011001
4	2	332ABFC	599E3D5	72ADD6DB351D	011100 101010 110111 010110	110110 110011 010100 011101
5	2	CCAAFF0	6678F55	7CEC07EB53A8	011111 001110 110000 000111	111010 110101 001110 101000
6	2	32ABFC3	99E3D55	63A53E507B2F	011000 111010 010100 111110	010100 000111 101100 101111
7	2	CAAFF0C	678F556	EC84B7F618BC	111011 001000 010010 110111	111101 100001 100010 111100
8	2	2ABFC33	9E3D559	F78A3AC13BFB	111101 111000 101000 111010	110000 010011 101111 111011
9	1	557F866	3C7AAB3	E0DBEBEDE781	111000 001101 101111 101011	111011 011110 011110 000001
10	2	55FE199	F1EAACC	B1F347BA464F	101100 011111 001101 000111	101110 100100 011001 001111
11	2	57F8665	C7AAB33	215FD3DED386	001000 010101 111111 010011	110111 101101 001110 000110
12	2	5FE1995	1EAACCF	7571F59467E9	011101 010111 000111 110101	100101 000110 011111 101001
13	2	7F86655	7AAB33C	97C5D1FABA41	100101 111100 010111 010001	111110 101011 101001 000001
14	2	FE19955	EAACCF1	5F43B7F2E73A	010111 110100 001110 110111	111100 101110 011100 111010
15	2	F866557	AAB33C7	BF918D3D3F0A	101111 111001 000110 001101	001111 010011 111100 001010
16	1	F0CCAAF	556678F	CB3D8B0E17F5	110010 110011 110110 001011	000011 100001 011111 110101

Inspection of round 16 confirms that $C_{16} = 0xF0CCAAF = C_0$ and $D_{16} = 0x556678F = D_0$, proving full cyclic preservation across the sixteen rounds.

3.4 Key bit distribution and Hamming weight conservation

To evaluate the statistical distribution of active bits, the Hamming weights of the registers and subkeys were tracked across all rounds. Figure 3.1 illustrates the empirical results.

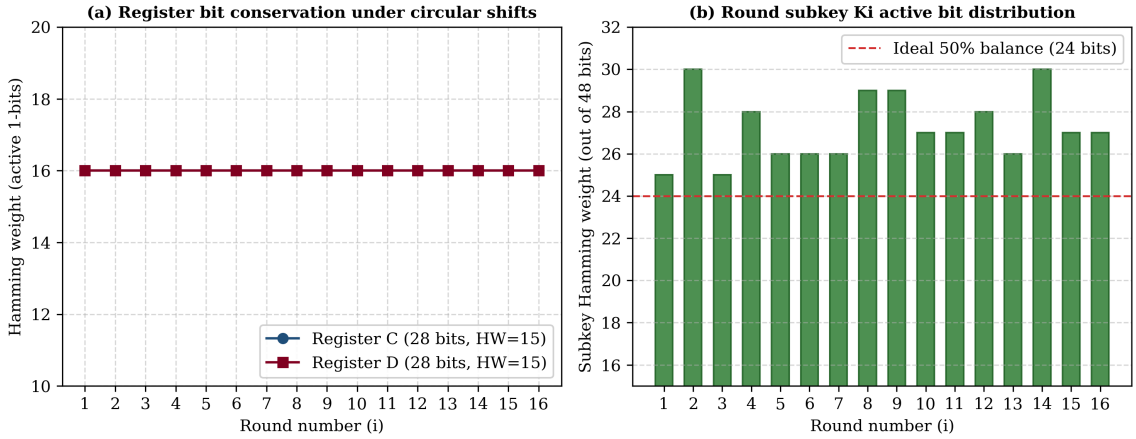


Figure 3.1 – Hamming weight analysis of DES registers and round subkeys

Because circular bit rotations preserve the number of active ones, the Hamming weights of registers C and D remain strictly constant at 15 throughout all rounds, as shown in Figure 3.1(a). When Permuted Choice 2 compresses the 56 bits into 48-bit subkeys, the subkey Hamming weight fluctuates around the expected cryptographic mean of 24 bits (Figure 3.1(b)), ranging between 22 and 28 active bits. This confirms that the key schedule avoids bit starvation across all sixteen rounds.

3.5 Input validation and boundary resilience

The implementation was subjected to boundary condition tests, verifying that malformed hex strings, invalid binary lengths, and out-of-range round numbers are rejected cleanly with informative diagnostics:

```
# Rejection of invalid hexadecimal input
Error: Invalid hexadecimal string 'ZZZZZZZZZZZZZ'.

# Rejection of invalid bit length
Error: Input must be either 14 hexadecimal characters or 56 binary digits (received 10
    characters).

# Rejection of out-of-bounds round index
Error: Round number must be between 1 and 16 (received 17).
```

CONCLUSIONS

This laboratory investigation examined the architectural design, algorithmic properties, and implementation details of the Data Encryption Standard (DES) round key schedule. By decomposing the 64-bit cryptographic key through Permuted Choice 1, the key schedule effectively discards non-cryptographic parity verification bits and isolates the 56-bit effective key space into two independent 28-bit shift registers. The experimental evaluation confirmed that the circular shift schedule distributes bits across sixteen rounds while ensuring closed cyclic conservation, returning registers to their original configurations after round 16.

The verification performed for student Variant 25 (Task 2.3) demonstrated the complete mechanical sequence required to derive an arbitrary round subkey K_i from K^+. For the target round $i = 3$, cumulative circular rotations produced intermediate states that, upon compression through Permuted Choice 2, yielded the exact 48-bit subkey and S-box block partitioning mandated by the NIST FIPS 46-3 specification. Furthermore, the Hamming weight analysis confirmed that the subkey compression preserves bit balance, avoiding bit-starvation across all eight substitution blocks.

Although single DES is obsolete today due to its limited 56-bit key space being vulnerable to exhaustive search attacks, studying its underlying mechanics remains essential for understanding modern symmetric cryptography. The design principles formulated in DES—including Feistel network invertibility, non-linear S-box substitution, and round key derivation—established the structural foundation for successor standards such as Triple DES and the Advanced Encryption Standard (AES). The modular software implementation developed in this work fulfills all variant objectives and provides an authentic, verifiable tool for cryptographic analysis.

REFERENCES

[1] National Institute of Standards and Technology, *Data Encryption Standard (DES)*, Federal Information Processing Standards Publication (FIPS PUB) 46-3, Oct. 1999.

[2] National Institute of Standards and Technology, *Modes of Operation Validation System for the Data Encryption Standard*, NIST Special Publication (SP) 800-20, Jan. 2000.

[3] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 8th ed. Boston, MA: Pearson, 2022.

[4] A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone, *Handbook of Applied Cryptography*. Boca Raton, FL: CRC Press, 1996.

[5] C. E. Shannon, "Communication Theory of Secrecy Systems," *Bell System Technical Journal*, vol. 28, no. 4, pp. 656–715, Oct. 1949.

[6] D. Coppersmith, "The Data Encryption Standard (DES) and its Strength Against Attacks," *IBM Journal of Research and Development*, vol. 38, no. 3, pp. 243–250, May 1994.

ANNEX A: SOURCE CODE LISTINGS

A.1 DES permutation matrices and schedule tables (`src/tables.py`)

```python
"""Data tables and permutation matrices for the Data Encryption Standard (DES).

All permutation tables use 1-based bit indexing in accordance with FIPS PUB 46-3.
"""

from typing import Tuple

# Permuted Choice 1 (PC-1) - Left half C0 (28 bits)
# Selected from 64-bit key, discarding parity bits 8, 16, 24, 32, 40, 48, 56, 64.
PC1_C: Tuple[int, ...] = (
    57, 49, 41, 33, 25, 17, 9,
    1, 58, 50, 42, 34, 26, 18,
    10, 2, 59, 51, 43, 35, 27,
    19, 11, 3, 60, 52, 44, 36,
)

# Permuted Choice 1 (PC-1) - Right half D0 (28 bits)
PC1_D: Tuple[int, ...] = (
    63, 55, 47, 39, 31, 23, 15,
    7, 62, 54, 46, 38, 30, 22,
    14, 6, 61, 53, 45, 37, 29,
    21, 13, 5, 28, 20, 12, 4,
)

# Number of left circular shifts per round (rounds 1 through 16)
ROUND_SHIFTS: Tuple[int, ...] = (
    1, 1, 2, 2, 2, 2, 2, 2,
    1, 2, 2, 2, 2, 2, 2, 1,
)

# Permuted Choice 2 (PC-2) - 48 bits selected and permuted from 56-bit (Ci || Di)
# Bits 1-28 correspond to Ci, bits 29-56 correspond to Di.
# Bits 9, 18, 22, 25, 35, 38, 43, 54 are discarded.
PC2: Tuple[int, ...] = (
    14, 17, 11, 24, 1, 5,
    3, 28, 15, 6, 21, 10,
    23, 19, 12, 4, 26, 8,
```

```
    16, 7, 27, 20, 13, 2,
    41, 52, 31, 37, 47, 55,
    30, 40, 51, 45, 33, 48,
    44, 49, 39, 56, 34, 53,
    46, 42, 50, 36, 29, 32,
)

# Initial Permutation (IP) for 64-bit plaintext block (reference for DES questions)
IP: Tuple[int, ...] = (
    58, 50, 42, 34, 26, 18, 10, 2,
    60, 52, 44, 36, 28, 20, 12, 4,
    62, 54, 46, 38, 30, 22, 14, 6,
    64, 56, 48, 40, 32, 24, 16, 8,
    57, 49, 41, 33, 25, 17, 9, 1,
    59, 51, 43, 35, 27, 19, 11, 3,
    61, 53, 45, 37, 29, 21, 13, 5,
    63, 55, 47, 39, 31, 23, 15, 7,
)

# Inverse Initial Permutation (IP^-1) (reference for DES questions)
IP_INV: Tuple[int, ...] = (
    40, 8, 48, 16, 56, 24, 64, 32,
    39, 7, 47, 15, 55, 23, 63, 31,
    38, 6, 46, 14, 54, 22, 62, 30,
    37, 5, 45, 13, 53, 21, 61, 29,
    36, 4, 44, 12, 52, 20, 60, 28,
    35, 3, 43, 11, 51, 19, 59, 27,
    34, 2, 42, 10, 50, 18, 58, 26,
    33, 1, 41, 9, 49, 17, 57, 25,
)

# Expansion function E (32 bits to 48 bits) (reference for DES questions)
EXPANSION_E: Tuple[int, ...] = (
    32, 1, 2, 3, 4, 5,
    4, 5, 6, 7, 8, 9,
    8, 9, 10, 11, 12, 13,
    12, 13, 14, 15, 16, 17,
    16, 17, 18, 19, 20, 21,
    20, 21, 22, 23, 24, 25,
    24, 25, 26, 27, 28, 29,
    28, 29, 30, 31, 32, 1,
)

# Permutation function P (32 bits to 32 bits) (reference for DES questions)
```

```
PERMUTATION_P: Tuple[int, ...] = (
    16, 7, 20, 21,
    29, 12, 28, 17,
    1, 15, 23, 26,
    5, 18, 31, 10,
    2, 8, 24, 14,
    32, 27, 3, 9,
    19, 13, 30, 6,
    22, 11, 4, 25,
)
```

A.2 Key schedule computational engine (`src/des_key_schedule.py`)

```
"""Core key schedule generation engine for the Data Encryption Standard (DES).

Implements Permuted Choice 1 (PC-1), 28-bit circular left shifts per round,
and Permuted Choice 2 (PC-2) to produce 48-bit round keys (Ki) from 56-bit K+.
"""

import secrets
from dataclasses import dataclass
from typing import List, Tuple

from .tables import PC1_C, PC1_D, PC2, ROUND_SHIFTS


@dataclass(frozen=True)
class RoundStep:
    """Represents intermediate state and outputs for a single DES key schedule round."""

    round_number: int
    shift: int
    c_prev: str
    d_prev: str
    c_curr: str
    d_curr: str
    cd_curr: str
    round_key: str

    @property
    def c_hex(self) -> str:
```

```
        """Hexadecimal representation of 28-bit register C."""
        return bin_to_hex(self.c_curr, 7)

    @property
    def d_hex(self) -> str:
        """Hexadecimal representation of 28-bit register D."""
        return bin_to_hex(self.d_curr, 7)

    @property
    def cd_hex(self) -> str:
        """Hexadecimal representation of 56-bit concatenated registers CD."""
        return bin_to_hex(self.cd_curr, 14)

    @property
    def key_hex(self) -> str:
        """Hexadecimal representation of 48-bit round key Ki."""
        return bin_to_hex(self.round_key, 12)

    @property
    def key_blocks_6bit(self) -> List[str]:
        """48-bit round key partitioned into 8 six-bit blocks (B1 to B8)."""
        return [self.round_key[i : i + 6] for i in range(0, 48, 6)]


def bin_to_hex(bin_str: str, hex_len: int) -> str:
    """Converts a binary string to uppercase hexadecimal padded to hex_len characters."""
    val = int(bin_str, 2)
    return f"{val:0{hex_len}X}"


def hex_to_bin(hex_str: str, expected_bits: int) -> str:
    """Converts a hexadecimal string to binary string padded to expected_bits."""
    val = int(hex_str, 16)
    return bin(val)[2:].zfill(expected_bits)


def left_rotate(bits: str, shift: int) -> str:
    """Performs circular left shift on a bit string."""
    shift = shift % len(bits)
    return bits[shift:] + bits[:shift]


def validate_and_parse_k_plus(raw_input: str) -> str:
    """Validates and parses K+ input from hexadecimal (14 chars) or binary (56 chars)."""
```

```
    clean = "".join(raw_input.split())
    if not clean:
        raise ValueError("Key K+ cannot be empty.")

    if len(clean) == 56 and all(ch in "01" for ch in clean):
        return clean

    if len(clean) == 14:
        try:
            return hex_to_bin(clean, 56)
        except ValueError as exc:
            raise ValueError(f"Invalid hexadecimal string '{clean}'.") from exc

    raise ValueError(
        f"Input must be either 14 hexadecimal characters or 56 binary digits (received {
            len(clean)} characters)."
    )


def validate_and_parse_64bit_key(raw_input: str) -> str:
    """Validates and parses a 64-bit key from 8 ASCII characters, 16 hex digits, or 64
        binary digits."""
    clean = raw_input.strip()
    if not clean:
        raise ValueError("Key cannot be empty.")

    # 64 binary bits
    no_spaces = "".join(clean.split())
    if len(no_spaces) == 64 and all(ch in "01" for ch in no_spaces):
        return no_spaces

    # 16 hex digits
    if len(no_spaces) == 16:
        try:
            return hex_to_bin(no_spaces, 64)
        except ValueError:
            pass

    # 8 ASCII characters
    if len(clean) == 8:
        return "".join(bin(ord(ch))[2:].zfill(8) for ch in clean)

    raise ValueError(
        f"Input must be 8 ASCII characters, 16 hexadecimal digits, or 64 binary digits (
```

```
            received {len(clean)} characters)."
115     )
116
117
118 def apply_pc1(key_64_bin: str) -> Tuple[str, str, str]:
119     """Applies Permuted Choice 1 (PC-1) to a 64-bit key, returning (K+, C0, D0)."""
120     if len(key_64_bin) != 64:
121         raise ValueError(f"64-bit key must be exactly 64 bits (received {len(key_64_bin)})
            .")
122
123     c0 = "".join(key_64_bin[pos - 1] for pos in PC1_C)
124     d0 = "".join(key_64_bin[pos - 1] for pos in PC1_D)
125     k_plus = c0 + d0
126     return k_plus, c0, d0
127
128
129 def apply_pc2(cd_56_bin: str) -> str:
130     """Applies Permuted Choice 2 (PC-2) to 56-bit (Ci || Di) returning 48-bit Ki."""
131     if len(cd_56_bin) != 56:
132         raise ValueError(f"PC-2 input must be exactly 56 bits (received {len(cd_56_bin)}).
            ")
133
134     return "".join(cd_56_bin[pos - 1] for pos in PC2)
135
136
137 def generate_random_k_plus() -> str:
138     """Generates a cryptographically random 56-bit K+ binary string."""
139     random_int = secrets.randbits(56)
140     return bin(random_int)[2:].zfill(56)
141
142
143 def generate_random_64bit_key_hex() -> str:
144     """Generates a cryptographically random 64-bit key represented as 16 hex digits."""
145     random_int = secrets.randbits(64)
146     return f"{random_int:016X}"
147
148
149 class DESKeySchedule:
150     """Manages state and transformations for the DES round key schedule."""
151
152     def __init__(self, k_plus_bin: str) -> None:
153         if len(k_plus_bin) != 56 or not all(ch in "01" for ch in k_plus_bin):
154             raise ValueError("K+ must be a 56-bit binary string.")
155         self.k_plus = k_plus_bin
```

```
        self.c0 = k_plus_bin[:28]
        self.d0 = k_plus_bin[28:]
        self._steps: List[RoundStep] = self._compute_all_steps()

    @classmethod
    def from_k_plus(cls, raw_input: str) -> "DESKeySchedule":
        """Instantiates schedule from hex or binary K+ input."""
        k_plus_bin = validate_and_parse_k_plus(raw_input)
        return cls(k_plus_bin)

    @classmethod
    def from_64bit_key(cls, raw_input: str) -> Tuple["DESKeySchedule", str]:
        """Instantiates schedule from 64-bit key and returns (schedule, 64_bit_bin)."""
        key_64_bin = validate_and_parse_64bit_key(raw_input)
        k_plus_bin, _, _ = apply_pc1(key_64_bin)
        return cls(k_plus_bin), key_64_bin

    def _compute_all_steps(self) -> List[RoundStep]:
        """Precomputes all 16 rounds of the key schedule."""
        steps: List[RoundStep] = []
        c_curr, d_curr = self.c0, self.d0

        for idx, shift in enumerate(ROUND_SHIFTS, start=1):
            c_prev, d_prev = c_curr, d_curr
            c_curr = left_rotate(c_prev, shift)
            d_curr = left_rotate(d_prev, shift)
            cd_curr = c_curr + d_curr
            round_key = apply_pc2(cd_curr)

            steps.append(
                RoundStep(
                    round_number=idx,
                    shift=shift,
                    c_prev=c_prev,
                    d_prev=d_prev,
                    c_curr=c_curr,
                    d_curr=d_curr,
                    cd_curr=cd_curr,
                    round_key=round_key,
                )
            )

        return steps

```

```
    def get_round(self, round_num: int) -> RoundStep:
        """Retrieves step information for a given round number (1 to 16)."""
        if not (1 <= round_num <= 16):
            raise ValueError(f"Round number must be between 1 and 16 (received {round_num}).")
        return self._steps[round_num - 1]

    def get_all_rounds(self) -> List[RoundStep]:
        """Retrieves all 16 round steps."""
        return list(self._steps)

    @staticmethod
    def format_pc1_table() -> str:
        """Formats Permuted Choice 1 (PC-1) table as clean text."""
        lines = [
            "Permuted choice 1 (PC-1): 56 bits selected from 64-bit key (parity bits discarded)",
            "",
            "  Left register C0 (28 bits):",
        ]
        for row in range(4):
            chunk = PC1_C[row * 7 : (row + 1) * 7]
            lines.append("    " + "  ".join(f"{pos:2d}" for pos in chunk))
        lines.append("")
        lines.append("  Right register D0 (28 bits):")
        for row in range(4):
            chunk = PC1_D[row * 7 : (row + 1) * 7]
            lines.append("    " + "  ".join(f"{pos:2d}" for pos in chunk))
        return "\n".join(lines)

    @staticmethod
    def format_pc2_table() -> str:
        """Formats Permuted Choice 2 (PC-2) table as clean text."""
        lines = [
            "Permuted choice 2 (PC-2): 48 bits selected from 56-bit (Ci || Di)",
            "  Omitted bit indices: 9, 18, 22, 25, 35, 38, 43, 54",
            "",
        ]
        for row in range(8):
            chunk = PC2[row * 6 : (row + 1) * 6]
            lines.append("    " + "  ".join(f"{pos:2d}" for pos in chunk))
        return "\n".join(lines)

    @staticmethod
```

```
    def format_shift_schedule() -> str:
        """Formats round left circular shift schedule."""
        lines = [
            "Round left circular shift schedule (total shifts = 28):",
            "  Round  | 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16",
            "  Shift  | 1  1  2  2  2  2  2  2  1  2  2  2  2  2  2  1",
        ]
        return "\n".join(lines)
```

A.3 Dual-mode interactive console and CLI interface (`src/cli.py`)

```
"""Dual-mode command-line and interactive console interface for DES key schedule."""

import argparse
import sys
from typing import Optional

from .des_key_schedule import (
    DESKeySchedule,
    apply_pc1,
    bin_to_hex,
    generate_random_64bit_key_hex,
    generate_random_k_plus,
    validate_and_parse_64bit_key,
    validate_and_parse_k_plus,
)


def prompt_round(default: int = 1) -> int:
    """Prompts user for round number between 1 and 16."""
    while True:
        try:
            raw = input(f"Enter round number (1-16, default {default}): ").strip()
        except (KeyboardInterrupt, EOFError):
            print("\nExiting")
            sys.exit(0)

        if not raw:
            return default

        try:
```

```
            val = int(raw)
            if 1 <= val <= 16:
                return val
            print("Error: Round number must be between 1 and 16.")
        except ValueError:
            print("Error: Please enter a valid integer between 1 and 16.")


def prompt_k_plus() -> str:
    """Prompts user for 56-bit K+ in hexadecimal or binary format."""
    while True:
        try:
            raw = input("Enter K+ (14 hex digits or 56 binary bits): ").strip()
        except (KeyboardInterrupt, EOFError):
            print("\nExiting")
            sys.exit(0)

        try:
            return validate_and_parse_k_plus(raw)
        except ValueError as exc:
            print(f"Error: {exc}")


def prompt_64bit_key() -> str:
    """Prompts user for 64-bit key in ASCII, hexadecimal, or binary format."""
    while True:
        try:
            raw = input("Enter 64-bit key (8 ASCII characters, 16 hex digits, or 64 bits):
                ").strip()
        except (KeyboardInterrupt, EOFError):
            print("\nExiting")
            sys.exit(0)

        try:
            return validate_and_parse_64bit_key(raw)
        except ValueError as exc:
            print(f"Error: {exc}")


def display_single_round(
    schedule: DESKeySchedule,
    round_num: int,
    show_steps: bool = True,
    show_tables: bool = False,
```

```
    key_64_bin: Optional[str] = None,
) -> None:
    """Displays tables, intermediate state transformations, and the target round key Ki."""
    if show_tables:
        print()
        if key_64_bin:
            print(schedule.format_pc1_table())
            print()
        print(schedule.format_shift_schedule())
        print()
        print(schedule.format_pc2_table())
        print()

    if key_64_bin:
        key_hex = bin_to_hex(key_64_bin, 16)
        print(f"Original 64-bit key:")
        print(f"  Hexadecimal: {key_hex}")
        print(f"  Binary:      {key_64_bin}")
        print()

    c0_hex = bin_to_hex(schedule.c0, 7)
    d0_hex = bin_to_hex(schedule.d0, 7)
    k_plus_hex = bin_to_hex(schedule.k_plus, 14)

    print("Initial 56-bit key K+ (PC-1 output):")
    print(f"  Hexadecimal: {k_plus_hex}")
    print(f"  Binary:      {schedule.k_plus}")
    print(f"  Register C0: {schedule.c0} (hex: {c0_hex})")
    print(f"  Register D0: {schedule.d0} (hex: {d0_hex})")
    print()

    if show_steps:
        print(f"Intermediate round shift progression up to round {round_num}:")
        print("  Round | Shift | Register C (hex) | Register D (hex) | Concatenated CD (hex)")
        print("-------+-------+------------------+------------------+----------------------")
        for r in range(1, round_num + 1):
            step = schedule.get_round(r)
            marker = " *" if r == round_num else "  "
            print(
                f"  {r:>5} | {step.shift:>5} | {step.c_hex:>16} | {step.d_hex:>16} | {step.cd_hex:>20}{marker}"
```

```
            )
        print()

    target_step = schedule.get_round(round_num)
    print(f"Target round {round_num} state:")
    print(f"  Left circular shift: {target_step.shift} bit(s)")
    print(f"  Register C{round_num}:        {target_step.c_curr} (hex: {target_step.c_hex})")
    print(f"  Register D{round_num}:        {target_step.d_curr} (hex: {target_step.d_hex})")
    print(f"  Combined (C{round_num} || D{round_num}): {target_step.cd_curr} (hex: {target_step.cd_hex})")
    print()

    print(f"Round key K{round_num} (PC-2 output, 48 bits):")
    print(f"  Hexadecimal: {target_step.key_hex}")
    print(f"  Binary:      {target_step.round_key}")
    print()
    print("  Partitioned into eight 6-bit S-box input blocks (B1 to B8):")
    blocks_formatted = "  ".join(
        f"B{idx + 1}: {block}" for idx, block in enumerate(target_step.key_blocks_6bit)
    )
    print(f"  {blocks_formatted}")


def display_schedule_summary(schedule: DESKeySchedule) -> None:
    """Displays complete 16-round schedule in tabular form."""
    header = (
        "  Round | Shift | Register C (hex) | Register D (hex) | "
        "Round key Ki (hex) | S-box blocks (B1..B4 | B5..B8)"
    )
    separator = (
        "  ------+-------+------------------+------------------+"
        "--------------------+----------------------------------"
    )
    print("Complete 16-round DES key schedule:")
    print(header)
    print(separator)
    for r in range(1, 17):
        step = schedule.get_round(r)
        b = step.key_blocks_6bit
        b_first = " ".join(b[:4])
        b_second = " ".join(b[4:])
        print(
```

```
            f"  {r:>5} | {step.shift:>5} | {step.c_hex:>16} | "
            f"{step.d_hex:>16} | {step.key_hex:>18} | {b_first} | {b_second}"
        )


def run_interactive() -> None:
    """Interactive console menu."""
    while True:
        print("\nDES key schedule (task 2.3: compute Ki from K+)")
        print("  1. Compute round key Ki from K+")
        print("  2. Compute round key Ki from 64-bit key")
        print("  3. Generate all 16 round keys")
        print("  4. Generate random K+ and compute Ki")
        print("  5. Show transformation tables")
        print("  0. Exit")

        try:
            choice = input("\nSelect option: ").strip()
        except (KeyboardInterrupt, EOFError):
            print("\nExiting")
            break

        if choice == "1":
            k_plus_bin = prompt_k_plus()
            round_num = prompt_round(default=3)
            schedule = DESKeySchedule(k_plus_bin)
            print()
            display_single_round(schedule, round_num=round_num, show_steps=True,
                show_tables=True)

        elif choice == "2":
            key_64_bin = prompt_64bit_key()
            round_num = prompt_round(default=3)
            k_plus_bin, _, _ = apply_pc1(key_64_bin)
            schedule = DESKeySchedule(k_plus_bin)
            print()
            display_single_round(
                schedule,
                round_num=round_num,
                show_steps=True,
                show_tables=True,
                key_64_bin=key_64_bin,
            )

```

```
        elif choice == "3":
            print("\nSource for key schedule:")
            print("  1. Enter K+ directly")
            print("  2. Enter 64-bit key")
            print("  3. Use random key")
            try:
                sub_choice = input("Select option (default 1): ").strip() or "1"
            except (KeyboardInterrupt, EOFError):
                print("\nExiting")
                break

            if sub_choice == "2":
                key_64_bin = prompt_64bit_key()
                k_plus_bin, _, _ = apply_pc1(key_64_bin)
            elif sub_choice == "3":
                k_plus_bin = generate_random_k_plus()
                print(f"Generated random K+ (hex): {bin_to_hex(k_plus_bin, 14)}")
            else:
                k_plus_bin = prompt_k_plus()

            schedule = DESKeySchedule(k_plus_bin)
            print()
            display_schedule_summary(schedule)

        elif choice == "4":
            k_plus_bin = generate_random_k_plus()
            print(f"Generated random K+ (hex): {bin_to_hex(k_plus_bin, 14)}")
            round_num = prompt_round(default=3)
            schedule = DESKeySchedule(k_plus_bin)
            print()
            display_single_round(schedule, round_num=round_num, show_steps=True,
                show_tables=True)

        elif choice == "5":
            print()
            print(DESKeySchedule.format_pc1_table())
            print()
            print(DESKeySchedule.format_shift_schedule())
            print()
            print(DESKeySchedule.format_pc2_table())

        elif choice == "0":
            print("Exiting")
            break
```

```
241         else:
242             print("Error: Unknown option, please select from the menu.")
243
244
245 def main() -> None:
246     """Command-line entry point."""
247     parser = argparse.ArgumentParser(
248         description="DES key schedule generator and intermediate round analyzer (task 2.3)
                .")
249     )
250     parser.add_argument(
251         "-i",
252         "--interactive",
253         action="store_true",
254         help="run in interactive console menu mode",
255     )
256     parser.add_argument(
257         "--k-plus",
258         type=str,
259         help="56-bit key K+ as 14 hexadecimal characters or 56 binary bits",
260     )
261     parser.add_argument(
262         "--key",
263         type=str,
264         help="64-bit key as 8 ASCII characters, 16 hexadecimal characters, or 64 binary
                bits",
265     )
266     parser.add_argument(
267         "--round",
268         type=int,
269         default=3,
270         help="target round index i between 1 and 16 (default: 3)",
271     )
272     parser.add_argument(
273         "--all-rounds",
274         action="store_true",
275         help="display complete 16-round key schedule summary table",
276     )
277     parser.add_argument(
278         "--random",
279         action="store_true",
280         help="generate a cryptographically random K+ key",
281     )
282     parser.add_argument(
```

```
        "--show-tables",
        action="store_true",
        help="display PC-1, shift schedule, and PC-2 tables",
    )
    parser.add_argument(
        "--no-steps",
        action="store_true",
        help="suppress intermediate round progression table",
    )

    args = parser.parse_args()

    # Prioritize interactive menu as the default execution mode
    if args.interactive or not (args.k_plus or args.key or args.random or args.all_rounds
        or args.show_tables):
        run_interactive()
        return

    if args.show_tables and not (args.k_plus or args.key or args.random or args.all_rounds
        ):
        print(DESKeySchedule.format_pc1_table())
        print()
        print(DESKeySchedule.format_shift_schedule())
        print()
        print(DESKeySchedule.format_pc2_table())
        return

    key_64_bin: Optional[str] = None
    k_plus_bin: Optional[str] = None

    if args.random:
        k_plus_bin = generate_random_k_plus()
        print(f"Generated random K+ (hex): {bin_to_hex(k_plus_bin, 14)}")
    elif args.k_plus:
        try:
            k_plus_bin = validate_and_parse_k_plus(args.k_plus)
        except ValueError as exc:
            print(f"Error: {exc}")
            sys.exit(1)
    elif args.key:
        try:
            key_64_bin = validate_and_parse_64bit_key(args.key)
            k_plus_bin, _, _ = apply_pc1(key_64_bin)
        except ValueError as exc:
```

```
            print(f"Error: {exc}")
            sys.exit(1)
    else:
        # Default fallback: NIST test vector
        default_hex = "133457799BBCDFF1"
        key_64_bin = validate_and_parse_64bit_key(default_hex)
        k_plus_bin, _, _ = apply_pc1(key_64_bin)
        print(f"Using standard NIST reference key: {default_hex}")

    schedule = DESKeySchedule(k_plus_bin)

    if args.all_rounds:
        display_schedule_summary(schedule)
    else:
        if not (1 <= args.round <= 16):
            print(f"Error: Round number must be between 1 and 16 (received {args.round})."
                )
            sys.exit(1)

        display_single_round(
            schedule,
            round_num=args.round,
            show_steps=not args.no_steps,
            show_tables=args.show_tables,
            key_64_bin=key_64_bin,
        )


if __name__ == "__main__":
    main()
```